
DPGrammar: Joint Viterbi Repair for Grammar-Constrained Diffusion Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

Incremental parsers are left-to-right; masked diffusion language models are not. A diffusion decoder commits tokens in confidence order, so a grammar can only be enforced at the parse frontier, and every token placed beyond it is an unvalidated fact that the parser may later reject. We present **DPGrammar**, a repair layer that treats such a rejection as a joint decision over the whole violated span rather than a single-token fix. The repair is a Viterbi search over (parser state $\times$ position) whose candidate set is drawn *from the automaton*—the tokens the parser can actually consume—rather than from the model’s top- k over the vocabulary. This distinction is not cosmetic: on JSONSchemaBench medium, 46% of live parser states admit fewer than 100 of 126,349 tokens, so ranking the vocabulary and filtering afterwards is rejection sampling against a small target and fails outright on 49.3% of repairs. Drawing from the automaton makes failure structurally impossible and takes the dead-end rate to 0%. On $n=511$ instances, DPGrammar reaches 96.7% schema validity against 91.8% for the same system without the DP layer and 78.3% for LAVE, the strongest published diffusion baseline, while collapsing mean repair events from 33.2 to 1.2. We further show that schema validity alone is a misleading objective: it scores `{"objects": []}` as a success and an 851-character document that does not quite close as a failure. Pairing validity with a content floor reverses the ranking between the two baselines and leaves DPGrammar first at every threshold.

1 Introduction

Masked diffusion language models [Nie et al., 2025, Ye et al., 2025] generate by iteratively unmasking positions in confidence order rather than left to right. This is what makes them fast, and it is also what makes grammar-constrained decoding hard. An incremental parser—the standard machinery for constrained decoding [Willard and Louf, 2023, Dong et al., 2024, Microsoft, 2025]—consumes one token at a time from the left. At any point during diffusion the sequence looks like

$$\underbrace{\text{[consumed prefix]}}_{\text{parser has validated}} \underbrace{\text{[frontier]}}_{\text{constrainable}} \text{[MASK]} \underbrace{\text{[token]}}_{\text{unvalidated}} \text{[MASK]} \dots$$

The grammar can be enforced only at the frontier. Everything the model has already placed beyond it was sampled without constraint, and—because the sampler is monotone, $x_0 \leftarrow \mathbf{1}[\text{mask}] \odot x_0 + (1 - \mathbf{1}[\text{mask}]) \odot x$ —it is already final. When the frontier advances into those positions, the parser may reject, and the decoder must repair a commitment it cannot take back.

Existing diffusion decoders handle this in one of three ways. They force near-autoregressive ordering, giving up the parallelism that motivates diffusion; they verify each frontier token by sampling

33 K random completions and abort a whole block on failure [Zhang et al., 2026]; or they remask the
 34 offending position and let the sampler try again. The third is cheap but myopic: it repairs one token
 35 at a time and cannot see that the violation is joint across several positions.

36 Contributions.

- 37 1. **DPGrammar**, a joint repair layer that solves the violated span with a Viterbi search over parser
 38 states (§4). On $n=511$ JSONSchemaBench medium instances it reaches 96.7% schema validity
 39 versus 91.8% without the layer, and reduces mean repair events from 33.2 to 1.2.
- 40 2. **Candidates from the automaton, not from the vocabulary** (§4.3). We show that proposing
 41 the model’s top- k and filtering by the grammar is rejection sampling against a target set that is
 42 usually tiny—46% of live states admit fewer than 100 of 126,349 tokens—and that drawing from
 43 the parser’s outgoing edge set instead makes repair failure structurally impossible (49.3% $\rightarrow$ 0%
 44 dead ends).
- 45 3. **A grammar-derived termination rule** (§4.4) that replaces a step-count threshold with the ques-
 46 tion “can the parser do anything other than stop?”, unifying JSON and SMILES under one rule
 47 with no per-dataset constant.
- 48 4. **A measurement correction** (§5). The completion flag commonly logged as `valid` is not schema
 49 validity; it overstates grammar-constrained methods by 4.6–6.7 pp and unconstrained ones by
 50 0 pp. We also show that validity alone rewards degenerate output, and report it against a content
 51 floor.

52 2 Related Work

53 **Constrained decoding for diffusion.** Mündler et al. [2025] propose the first CFG-constrained
 54 dLLM decoder. DINGO [Suresh et al., 2025] applies dynamic programming for syntactic correct-
 55 ness at a higher per-step cost, over the full sequence rather than over a violated span. LAVE [Zhang
 56 et al., 2026] verifies each frontier token by sampling K random completions, accepting only if one
 57 is grammar-valid; it is the dominant published baseline. Plan-style decoders [Li et al., 2026] empha-
 58 sise structure-aware planning. We stay at the token level with an explicit incremental matcher, and
 59 add joint search only where the parse actually conflicts.

60 **Constrained decoding for autoregressive models.** Outlines [Willard and Louf, 2023], XGram-
 61 mar [Dong et al., 2024] and llguidance [Microsoft, 2025] compile grammars into incremental au-
 62 tomata and mask logits per step. All assume left-to-right commitment; the question of what to do
 63 when a token is committed *out of order* does not arise, and it is precisely the question diffusion
 64 raises.

65 **Speed.** Speculative decoding [Leviathan et al., 2023] and sparse automaton acceleration [Su et al.,
 66 2026] reduce the cost of verification. Our repair layer is complementary: it fires on roughly one
 67 position per instance and is not on the critical path of a clean step.

68 3 Problem Setting

69 Let G be a grammar with an incremental matcher exposing four operations: `try_consume`,
 70 `rollback`, `clone`, and a mask $\mathcal{L}(q) \subseteq V$ of tokens consumable from state q . Let the decoder
 71 hold a sequence x over $V \cup \{\text{MASK}\}$ and a frontier index c such that $x_{<c}$ has been consumed,
 72 leaving the matcher in state q_c .

73 At each denoising step the model proposes tokens at masked positions. Frontier masking guarantees
 74 $x_c \in \mathcal{L}(q_c)$ when x_c is chosen this step, but positions $> c$ are unconstrained. After placement
 75 the decoder advances the parser over the newly contiguous region; if it stops at position v with
 76 $x_v \notin \mathcal{L}(q_v)$, we call v a *violation*.

77 **Repair as search.** The repair problem is: find y over a span $[v, e)$ maximising $\sum_p \log p_\theta(y_p | x)$
 78 subject to the matcher accepting $y_v \cdots y_{e-1}$ from q_v . This is a shortest-path problem on a layered
 79 graph whose nodes are (position, parser state) and whose edges out of a state are exactly $\mathcal{L}(q)$.

Two design choices determine whether the search is sound, and both were wrong in the natural implementation.

4 Method

4.1 The repair cascade

When the parser rejects at v , the decoder tries, in order:

1. **Termination check.** If the parser cannot consume anything, or the model has placed a stop token at the frontier and the parser permits stopping, the document ends (§4.4).
2. **Single-token retry.** Walk the ranked logits at v for up to 10 candidates and take the first the parser accepts. This resolves 62% of violations at negligible cost.
3. **Joint DP repair.** Otherwise solve the span $[v, e)$ jointly (§4.2).
4. **Hand back.** If no assignment exists, remask v and let the sampler re-derive it.

The span end e is $\min(v + L, \text{first MASK})$ with L a compute budget; the DP does not fill holes, only rewrites committed positions.

4.2 Joint repair by Viterbi over parser states

We maintain a map from parser state to the best path reaching it:

$$S_p : q \mapsto (\text{matcher clone}, \text{edits}, \sum \log p_\theta).$$

Two paths that reach the same state are merged, keeping the higher-scoring one; this is what makes the search a dynamic program rather than an enumeration, and it is sound because the state determines the future. At the final layer we take the highest-scoring state and backtrack.

The state key is the matcher’s own mask, $\text{bytes}(\mathcal{L}(q))$, which the implementation already computes; distinct keys are distinct automaton nodes. In practice $|S_p|$ stays in the single digits, so the cost is $O(\text{span} \times |S| \times k)$ matcher operations with $|S|$ bounded by the reachable automaton rather than by k^{span} .

4.3 Candidates from the automaton

The natural implementation asks the model for its top- k tokens and then asks the parser which of them it accepts. This is rejection sampling against $\mathcal{L}(q)$, and it fails whenever $\mathcal{L}(q)$ is small relative to $|V|$. Measuring 1,687 live states on JSONSchemaBench medium with $|V| = 126,349$, the distribution of $|\mathcal{L}(q)|$ is sharply bimodal:

$$p_{25} = 94, \quad p_{50} = 101, \quad p_{75} = 125,311,$$

with 46% of states admitting fewer than 100 tokens. At those states—key boundaries, enumerations, closings—a top-100 draw from a 126k vocabulary frequently misses $\mathcal{L}(q)$ entirely, and the DP reports “no path” where a path exists. At a freshly opened JSON object we measured $|\mathcal{L}(q)| = 2$.

We therefore invert the order. For each live state we read $\mathcal{L}(q)$ and take the k most probable tokens *within it*:

$$\mathcal{C}(q, p) = \text{top-}k_{t \in \mathcal{L}(q) \setminus \{\text{EOS}\}} \log p_\theta(t \mid x, p). \quad (1)$$

Since a live state always admits at least one token—0 of 2,028 sampled states had an empty mask—the candidate set is never empty and the DP can always complete the span. Where $|\mathcal{L}(q)| \leq k$ the edge set is enumerated exactly rather than sampled; this holds at 67% of state-positions. Empirically the dead-end rate falls from 49.3% to 0%, repair events fall by 46%, and wall time falls slightly. The cost is one mask read per (state, position), which the implementation already paid to compute state keys.

Why EOS is excluded. The stop token is marked legal by the mask at accepting states but is not a grammar symbol: `try_consume` refuses it, because there is nothing to advance over. Offering it wastes a candidate slot and, where it is the only “legal” token, empties the set and imitates a dead end.

122 4.4 Termination as a grammar question

123 Because EOS is not consumable, a finished document arrives at the frontier looking like a violation,
124 and something must recognise it. The natural test—"is the parser in an accepting state?"—is wrong,
125 and the error is invisible on JSON. It asks whether the parser *may* stop, not whether it *must*. On 53
126 of 53 completed JSON documents the accepting state admitted exactly one token, EOS, so the two
127 questions coincide. On SMILES a complete molecule such as CCO is accepting with 1,388 tokens
128 still legal, and stopping there takes the length decision away from the model: every molecule came
129 out as C or CC.

130 We use two rules, for two different decisions:

- 131 • *The harness ends the document* only when $\mathcal{L}(q) \setminus \{\text{EOS}, \text{EOT}\} = \emptyset$ and q is accepting—the parser
132 has no alternative, so ending is not a choice anyone is making.
- 133 • *The model ends the document* when it places a stop token at the frontier and q is accepting—the
134 model has asked, and the grammar need only permit it.

135 This replaces a step-count threshold with a property of the grammar. On JSON it reproduces the
136 previous behaviour exactly; on SMILES it recovers median output length $3 \rightarrow 8$ characters and
137 RDKit validity $62.5\% \rightarrow 81.2\%$ while eliminating all repair activity (16 DP calls and 16 remarks $\rightarrow$
138 0).

139 4.5 System optimisations

140 The layer sits on a frontier-constrained decoder whose engineering we treat as infrastructure rather
141 than contribution: an incremental matcher carried across denoising steps instead of re-parsing the
142 prefix, additive-increase batching of committed tokens, in-place resampling that reuses the current
143 logits, and overlapping mask construction with the next forward pass. Throughout the paper, "**no-**
144 **DP baseline**" denotes exactly this system with the repair layer disabled, so that every comparison
145 isolates the DP layer.

146 5 Evaluation Methodology

147 5.1 The completion flag is not validity

148 Drivers in this setting commonly log a boolean `valid` taken from the generator, defined as "a stop
149 token is present and no position before it is masked". That is *completion*, not validity: an uncon-
150 strained decoder can finish cleanly on malformed JSON and score a success. Recomputing schema
151 validity with `jsonschema` over the stored schemas gives

	completion flag	schema validity
152 LAVE	78.3%	78.3%
No-DP baseline	98.4%	91.8%
DPGrammar	96.7%	96.7%

153 The bias is not uniform. Grammar-constrained methods emit documents that terminate cleanly but
154 are structurally broken—one closing brace too many, a key name written over a value—which count
155 as complete but not valid. LAVE falls back to autoregressive decoding on failure and produces well-
156 formed JSON, so for it the two notions coincide. Any comparison that mixes the two measures the
157 fallback strategy rather than the constraint algorithm. All numbers we report are recomputed schema
158 validity.

159 5.2 Validity alone rewards giving up

160 Schema validity is monotone in nothing that a reader cares about. On in-
161 stance `o17539` the no-DP baseline emits `{"objects": []}`—zero leaves, schema-
162 valid, scored a win—while DPGrammar emits 851 characters of populated objects
163 that do not quite close, scored a loss. On `o11975` the baseline’s "success" is

Table 1: JSONSchemaBench medium, $n=511$. Validity is recomputed with jsonschema; “ κ ” columns additionally require κ scalar leaves. Repair events are remarks for the grammar-based methods and lookahead draws for LAVE.

	LAVE	No-DP baseline	DPGrammar
Schema validity	78.3%	91.8%	96.7%
valid $\wedge \kappa=1$	78.3%	84.9%	93.7%
valid $\wedge \kappa=3$	76.7%	79.1%	90.6%
valid $\wedge \kappa=5$	74.0%	74.6%	84.9%
valid $\wedge \kappa=10$	56.0%	50.7%	60.5%
valid $\wedge \kappa=15$	33.7%	29.5%	35.0%
Median leaves (valid only)	13	11	12
Median chars (valid only)	530	385	466
Mean repair events	253.46	33.23	1.19
Timeouts (> 120 s)	60	0	0
Mean time (s)	38.50	13.45	16.75
Median time (s)	25.70	13.40	15.05
P95 time (s)	120.00	21.43	36.21

164 {"faction": "rebel", "pilots": [{"name": "0", "ship": "0"}], "version": "0.0.0"}.
165 Across the benchmark, 22% of the baseline’s valid outputs carry fewer than five leaves.
166 We therefore report validity against a content floor: an output counts only if it validates *and* carries
167 at least κ scalar leaves. We report $\kappa \in \{1, 3, 5, 10, 15\}$ so that the threshold is visibly not doing the
168 work.

169 5.3 Setup

170 LLaDA-8B-Instruct [Nie et al., 2025], $T=128$ denoising steps, generation length 256, block length
171 32, temperature 0.2, on a single A100 per shard. Grammars are compiled by llguidance [Microsoft,
172 2025]. JSONSchemaBench [Geng et al., 2025] medium provides 586 instances; 75 have schemas ll-
173 guidance cannot compile, so no method runs on them and they are excluded from every arm, leaving
174 $n=511$. LAVE is reimplemented on the same backbone and step budget with a 120 s per-instance
175 timeout.

176 6 Results

177 **The DP layer is what buys content, not only validity.** Table 1 shows the no-DP baseline ahead
178 of LAVE on raw validity by 13.5 pp. The lead shrinks monotonically as the content floor rises—
179 to 6.6 pp at $\kappa=1$, 2.4 pp at $\kappa=3$, and 0.6 pp at $\kappa=5$ —and reverses beyond it, to -5.3 pp at $\kappa=10$
180 and -4.2 pp at $\kappa=15$. Median length of its valid outputs is 385 characters against LAVE’s 530. A
181 substantial part of the baseline’s apparent advantage is bought by emitting the smallest document
182 the schema permits, and the advantage disappears exactly when the metric stops accepting such
183 documents.

184 DPGrammar does not make that trade. It is first at every threshold, and its valid outputs are longer
185 and richer than the baseline’s (466 characters, 12 leaves versus 385 and 11). This is the result the
186 raw-validity table conceals: the DP layer’s contribution is not a few more valid documents, it is valid
187 documents that still say something.

188 **Cost.** Mean repair events fall from 33.2 to 1.19, a $28\times$ reduction, for a 3.3 s increase in mean wall
189 time. LAVE’s stochastic verification costs 253.5 draws per instance and times out on 60 of 511; both
190 grammar-based methods time out on none.

7 Analysis

7.1 What the repair layer actually changes

We instrumented every repair site. Of 118 violations on a 64-instance subsample, 62% are resolved by the single-token retry and never reach the DP. Of those that do, the automaton candidate set eliminates dead ends entirely, and the DP writes a mean of 5.95 tokens over a mean span of 14.0.

7.2 Ablations, including four that changed nothing

We report these because they delimit where the gain does and does not come from.

Component	Varied	Effect on validity
Candidate source	vocabulary top- $k \rightarrow$ automaton	dead ends 49.3% $\rightarrow$ 0%
Repair window	single token $\rightarrow$ full span	1-char leaves 22.3% $\rightarrow$ 7.3%
Termination rule	accepting $\rightarrow$ nothing-consumable	SMILES 62.5% $\rightarrow$ 81.2%
Objective	$\log p \rightarrow$ min-edit	none (−0.8 edits)
Candidate admission	always admit committed token	none (identical output)
Post-hoc enrichment	on $\rightarrow$ off	none (identical output)
Edit penalty	$\lambda \in \{0, 2\}$	none (never non-zero)

The objective does not matter; the search space does. Replacing $\log p$ with a lexicographic minimum-edit objective reduces mean edits per repair only from 5.95 to 5.15. Since minimum-edit attains the minimum by construction, 87% of the rewriting the DP performs is *forced* by the grammar given the violation, not chosen by the objective. Changing what the search optimises therefore cannot help; changing what it searches over—the candidate set—moves the dead-end rate from 49.3% to 0.

7.3 What repair cannot fix

Ilguidance compiles a JSON schema into a grammar that fixes the order of an object’s properties to their declaration order: `{"a": 1, "b": 2}` is rejected when the schema declares `b` first, although both orders satisfy the schema and `jsonschema` accepts either. A model that writes the properties in a different order therefore produces a violation that is not an error, and the correct repair—moving a block—is not expressible by a positional DP, which can only overwrite in place. The repair then writes the demanded key name over a value intended for another field. We measured this class at 8% of violations; at states where the grammar admits exactly one token, 56% of violations are of this kind.

8 Limitations

Metric threshold. The content floor κ is a reporting choice, not a learned quantity. We report five values and the ranking of DPGrammar is stable across all of them, but the two baselines exchange places between $\kappa=3$ and $\kappa=5$, so any single-threshold claim about them would be fragile.

Scope. Results are one backbone (LLaDA-8B-Instruct), one benchmark split, and one seed. The SMILES evidence in §4.4 is a 16-instance pilot and establishes only that the termination rule generalises across grammar formalisms, not that the validity gain does.

Remaining failures. Of the residual failures we inspected, all were grammar-valid prefixes: the documents were incomplete rather than wrong, so the repair layer had nothing to repair. Their causes are the generation budget and the completion path, not the constraint algorithm.

Compute budget. L (span cap) and k (candidates per state) are compute budgets, not tuned constants; we did not sweep them. With automaton candidates k no longer controls whether an answer is found, only how many legal alternatives are weighed, so the search is not silently truncated by the choice.

9 Conclusion

Grammar-constrained decoding for diffusion models is a repair problem, and repair is a search over the parser’s own transition structure. Drawing candidates from that structure rather than from the model’s vocabulary ranking removes repair failure as a category; solving the violated span jointly rather than one token at a time removes the degeneracy that single-token repair induces; and deriving termination from the grammar rather than from a step count makes the same rule work on JSON and SMILES. The resulting system reaches 96.7% schema validity with 1.2 repair events per instance, and—unlike the baselines—does not buy validity by emitting less.

References

- Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems* 7, 2024.
- Saibo Geng, Hudson Cooper, Micha Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Jsonschemabench: A rigorous benchmark of structured outputs for language models, 2025. URL <https://arxiv.org/abs/2501.10868>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023. URL <https://arxiv.org/abs/2211.17192>.
- Miao Li, Hanyang Jiang, Sikai Cheng, Hengyu Fu, Yuhang Cai, Baihe Huang, Tinghan Ye, Xuanzhou Chen, and Pascal Van Hentenryck. Plan, verify and fill: A structured parallel decoding approach for diffusion language models, 2026. URL <https://arxiv.org/abs/2601.12247>.
- Microsoft. LLMGuidance: Fast structured outputs for LLMs. GitHub repository, 2025. URL <https://github.com/guidance-ai/llguidance>.
- Niels Mündler, Jasper Dekoninck, and Martin Vechev. Constrained decoding of diffusion llms with context-free grammars, 2025. URL <https://arxiv.org/abs/2508.10111>.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Jirong Wen, and Chongxuan Li. Large language diffusion models, 2025. URL <https://arxiv.org/abs/2502.09992>.
- Zhengyang Su, Isay Katsman, Yueqi Wang, Ruining He, Lukasz Heldt, Raghunandan Keshavan, Shao-Chuan Wang, Xinyang Yi, Mingyan Gao, Onkar Dalal, Lichan Hong, Ed Chi, and Ningren Han. Vectorizing the trie: Efficient constrained decoding for llm-based generative retrieval on accelerators, 2026. URL <https://arxiv.org/abs/2602.22647>.
- Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. Dingo: Constrained inference for diffusion llms, 2025. URL <https://arxiv.org/abs/2505.23061>.
- Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025. URL <https://arxiv.org/abs/2508.15487>.
- Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. Lookahead-then-verify: Reliable constrained decoding for diffusion llms under context-free grammars, 2026. URL <https://arxiv.org/abs/2602.00612>.